Swinburne University of Technology

School of Science, Computing and Engineering Technologies

ASSIGNMENT COVER SHEET

Subject Code: COS30008
Subject Title: Data Structures and Patterns
Assignment number and title: 1, Solution Design in C++
Due date: Sunday, March 30, 2025, 23:59
Lecturer: Dr. Markus Lumpe

Your name: Chrishelle Anthoneyz **Your student ID:** 104067504

Marker's comments:

Problem	Marks	Obtained
1	38	
2	170	
Total	208	

Extension certification:

This assignment has been given an extension and is now due on _____

Signature of Convener:_____

```cpp
#include "Vector3D.h"
#include <sstream>
#include <cmath>

// == operator for Vector3D: compares each component using epsilon
bool Vector3D::operator==(const Vector3D& aOther) const noexcept {
    return (std::fabs(x() - aOther.x()) < eps) &&
           (std::fabs(y() - aOther.y()) < eps) &&
           (std::fabs(w() - aOther.w()) < eps);
}

// toString(): returns a textual representation in the format "[x,y,w]"
std::string Vector3D::toString() const noexcept {
    std::stringstream ss;
    ss << "[" << x() << "," << y() << "," << w() << "]";
    return ss.str();
}

```

File - C:\ds\assignement\Matrix3x3_PS1.cpp

```cpp
#include "Matrix3x3.h"
#include <sstream>
#include <cassert>
#include <cmath>

// Matrix equivalence: Two matrices are equal if all their corresponding row vectors are equal.
bool Matrix3x3::operator==(const Matrix3x3& aOther) const noexcept {
    return (fRows[0] == aOther.fRows[0]) &&
           (fRows[1] == aOther.fRows[1]) &&
           (fRows[2] == aOther.fRows[2]);
}

// Matrix multiplication: Each element (i, j) in the result is the dot product of the i-th row of this matrix and the j-th column of aOther.
Matrix3x3 Matrix3x3::operator*(const Matrix3x3& aOther) const noexcept {
    const Matrix3x3& M = *this;
    return Matrix3x3(
        Vector3D( M[0].dot(aOther.column(0)), M[0].dot(aOther.column(1)), M[0].dot(aOther.column(2)) ),
        Vector3D( M[1].dot(aOther.column(0)), M[1].dot(aOther.column(1)), M[1].dot(aOther.column(2)) ),
        Vector3D( M[2].dot(aOther.column(0)), M[2].dot(aOther.column(1)), M[2].dot(aOther.column(2)) )
    );
}

// Transpose: Create a new matrix where each row is a column of the original matrix.
Matrix3x3 Matrix3x3::transpose() const noexcept {
    return Matrix3x3(
        column(0),
        column(1),
        column(2)
    );
}

// Determinant: Compute the determinant using the standard 3x3 formula.
float Matrix3x3::det() const noexcept {
    const Matrix3x3& M = *this;
    float M11 = M[0][0], M12 = M[0][1], M13 = M[0][2];
    float M21 = M[1][0], M22 = M[1][1], M23 = M[1][2];
    float M31 = M[2][0], M32 = M[2][1], M33 = M[2][2];
    return M11 * (M22 * M33 - M23 * M32)
```

```
39           - M12 * (M21 * M33 - M23 * M31)
40           + M13 * (M21 * M32 - M22 * M31);
41 }
42
43 // Invertibility check: A matrix is invertible if the absolute
   value of its determinant is greater than eps.
44 bool Matrix3x3::hasInverse() const noexcept {
45     return std::fabs(det()) > eps;
46 }
47
48 // Inverse:
49 // First compute the cofactors
50 // Then form the transpose of the cofactor matrix
51 // After that multiply by 1/det.
52 Matrix3x3 Matrix3x3::inverse() const noexcept {
53     const Matrix3x3& M = *this;
54     float determinant = det();
55     assert(std::fabs(determinant) > eps); // assert if the
   matrix is invertible.
56
57     // Compute cofactors (with alternating signs)
58     float c00 =  (M[1][1] * M[2][2] - M[1][2] * M[2][1]);
59     float c01 = -(M[1][0] * M[2][2] - M[1][2] * M[2][0]);
60     float c02 =  (M[1][0] * M[2][1] - M[1][1] * M[2][0]);
61
62     float c10 = -(M[0][1] * M[2][2] - M[0][2] * M[2][1]);
63     float c11 =  (M[0][0] * M[2][2] - M[0][2] * M[2][0]);
64     float c12 = -(M[0][0] * M[2][1] - M[0][1] * M[2][0]);
65
66     float c20 =  (M[0][1] * M[1][2] - M[0][2] * M[1][1]);
67     float c21 = -(M[0][0] * M[1][2] - M[0][2] * M[1][0]);
68     float c22 =  (M[0][0] * M[1][1] - M[0][1] * M[1][0]);
69
70     // Adjugate is the transpose of the cofactor matrix.
71     Matrix3x3 adjugate(
72         Vector3D(c00, c10, c20),
73         Vector3D(c01, c11, c21),
74         Vector3D(c02, c12, c22)
75     );
76
77     return adjugate * (1.0f / determinant);
78 }
79
80 // Output operator: Format the matrix as [[row1],[row2],[row3
   ]] using Vector3D's toString() method
81 std::ostream& operator<<(std::ostream& aOStream, const
   Matrix3x3& aMatrix) {
82     aOStream << "["
```

```
83                << aMatrix[0].toString() << ","
84                << aMatrix[1].toString() << ","
85                << aMatrix[2].toString() << "]";
86      return aOStream;
87  }
88
```